

to the serial ports. We need to stop it from /boot/cmdline.txt file. This is a plain text file, parsed by Raspi as a text file. You need to remove a line from cmdline.txt file by issuing the following command. (You need to have a backup file of this, so save the original file as cmdline_backup.txt.)

```
$sudo nano /boot/cmdline.txt
```

Remove console=ttyAMA0, 115200 kgdboc=ttyAMA0, 115220 line.

Save and exit. Reboot your Raspi so that everything comes into effect.

EFY Note

The source codes of this project are included in this month's EFY DVD and are also available for free download at source.efymag.com

Arduino-Raspi connection

Arduino pin	Raspi pin
TX	10 (through resistor network as shown in Fig. 3)
RX	Pin 8 (direct)
Gnd	Pin 6 or pin 9 or pin 20 (direct)

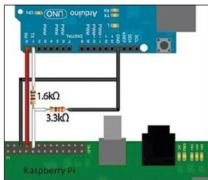


Fig. 3: Connection between Arduino and Raspi

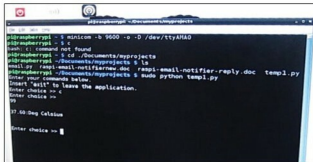


Fig. 4: Temperature reading on the terminal

Piggyback configuration

Arduino will sit on top of Raspi and then interact with the LM35 sensor. Arduino will give data to the serial port of Raspi.

GPIO 14 and GPIO 15 (physical pins 8 and 10) of Raspi are UART TX and RX pins. RX and TX pins of Arduino will just go into these GPIO 14 and 15 pins and the ground pin of Arduino will go to any one ground pin (physical pin 6, 9 or 20) of Raspi.

But, do not do that yet. Your Raspi board will be damaged beyond repair. The voltage level of Arduino output pin is approximately 5V, whereas that of Raspi is hardly 3V. We need to reduce Arduino I/O to the matching voltage level of Raspi using the circuit shown in Fig. 3.

Only three wires are to be connected, rest is the power supply that can be separate or common. Serial connection wires, TX and RX, will be interchanged with each other using a resistor network, as shown in the table, on Arduino TX pin to Raspi RX pin (pin 10).

Testing the project

Now, our main intention of making Arduino piggyback on Raspi is on the way. After connecting Arduino and Raspi, as shown in the table and Fig. 3, switch on the power supply of both boards.

Connect Raspi either through a headless mode or in direct mode (I always prefer the headless mode) and then run Minicom as follows:

```
$ ssh -Y pi@192.168.1.4
```

```
$ minicom -b 9600 -o -D /dev/ttyAMA0
```

Minicom has a help window that can be accessed by pressing Ctrl+a, z command to see more detail on Minicom.

Minicom will fetch the analogue data from serial port of Raspi and display it on the terminal

window. So, by using the command line and Minicom, we can bridge Arduino and Raspi boards for our application. To achieve this, we use Python program temp1.py to access the serial port of Raspi.

```
# temp1.py
import serial
import time

ser = serial.Serial('/dev/ttyAMA0', 9600,
    timeout=10)
ser.open()
#ser.write("c")

print 'Enter your commands below.\r\n'
    ninsert "exit" to leave the
    application.'

input=1
while 1 :
    # get keyboard input
    input = raw_input("Enter choice >> ")
    # Python 3 users
    # input = input(">> ")
    if input == 'exit':
        ser.close()
        exit()
    else:
        ser.write(input)

    out = ''
    time.sleep(1)
    while ser.inWaiting() > 0:
        out += ser.read(1)
    if out != '':
        print "Enter choice >>" + out
```

Run the Python program by issuing following command:

```
$sudo python temp1.py
```

As the serial console opens in Raspi, enter c to get temperature in Celsius or f to get temperature in Fahrenheit.

This is how you will be able to see temperature data coming from LM35 connected to Arduino, which is sitting piggyback on Raspi. The temperature output reading on the terminal taken during testing is shown in Fig. 4. ●



Somnath Bera is an avid user of open source software. Professionally, he is a thermal power expert working as additional general manager at NTPC Ltd